

Exercice 1 (7*1=7 pts)

1. On dispose d'un fichier nommé `data.txt` contenant les lignes suivantes :

```
Orange:2
Pomme:5
Banane:6
Orange:9
Kiwi:5
Orange:8
```

a. Écrire une commande qui compte le nombre total de lignes du fichier.

\$ wc -l data.txt

b. Copier les quantités des fruits dans un fichier `quantite.txt`

cut -d: -f2 data.txt > quantite.txt

c. Remplacer **toutes les lettres majuscules** par des **minuscules** dans ce fichier, le trier par ordre alphabétique et supprimer les doublons. Si l'exécution de la commande est correcte, la rediriger vers un fichier `sortie.txt` sinon vers un fichier `erreur.txt`.

\$ Cat data.txt | tr [A-Z] [a-z] | sort | uniq 1> sortie.txt 2> erreur.txt

2. Écrire une commande qui extrait de la 10ème ligne au 20ème ligne d'un fichier `logfile.txt` et les stocke triées dans un fichier `summary.txt`.

\$ head -n 20 logfile.txt | tail -n 10 | sort > summary.txt

3. Fusionner `file1.txt` et `file2.txt` ligne par ligne en séparant les colonnes par ; (ce qui donne : `ligne1` de `file1`; `ligne1` de `file2`, etc) et enregistrer le résultat dans `merged.txt`.

\$paste -d ";" file1.txt file2.txt > merged.txt

4. Ecrire une commande `grep` qui sélectionne toutes les lignes d'un fichier `f.txt` et qui vérifient à la fois toutes ces conditions :

- Commençant par une majuscule et,
- Contenant exactement 4 chiffres consécutifs
- Finissant par un point ou un point d'exclamation.

\$grep '^[A-Z].*[0-9]\{4\}.*[!]' f.txt

5. Que fait la commande suivante : `grep "/bin/bash" /etc/passwd | cut -d: -f1`

Elle affiche les noms des utilisateurs utilisant /bin/bash comme shell

Exercice 2 (3+2+2+1+1=9 pts)

Le fichier `ventesSupermarche.txt` contient des informations sur les ventes réalisées dans un supermarché. Chaque ligne du fichier contient les informations suivantes :

`date, catégorie_produit, produit, quantité_vendue, prix_unitaire, mode_paiement`

Exemple de quelques lignes du fichier :

```
15/04/2025,boissons,soda,10,1.50,carte
15/04/2025,produits_laitiers,lait,5,0.80,espèces
15/04/2025,fruits,pommes,8,0.60,carte
16/04/2025,boissons,eau,12,1.00,carte
16/04/2025,produits_laitiers,yaourt,6,1.20,espèces
16/04/2025,fruits,bananes,10,0.50,espèces
```

1. Ecrire le script `verif_format.sh` qui :

- i. Vérifie qu'on lui passe bien un fichier en argument, et que ce fichier existe, sinon affiche un message d'erreur,
- ii. Vérifie la validité du format de chaque ligne en affichant le numéro des lignes invalides,
- iii. Compte et affiche le nombre de lignes invalides.

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 fichier.csv"
    exit 1
fi

fichier="$1"
if [ ! -f "$fichier" ]; then
    echo "Erreur : fichier '$fichier' introuvable."
    exit 1
fi

awk -F',' '
BEGIN {
    erreurs = 0
}
{
    if (!($1 ~ /^[0-9]{2}\/[0-9]{2}\/[0-9]{4}$/ &&
        $2 ~ /^[^,]+$ / &&
        $3 ~ /^[^,]+$ / &&
        $4 ~ /^[0-9]+$ / &&
        $5 ~ /^[0-9]+\.[0-9]{2}$/ &&
        $6 ~ /^[^,]+$ /)) {
        print "Ligne " NR " invalide : " $0
        erreurs++
    }
}
```

```

    }
}
END {
    if (erreurs == 0)
        print "Toutes les lignes sont valides."
    else
        print "" erreurs " ligne(s) invalide(s) détectée(s)."
    }
}
' "$fichier"

```

2. En utilisant **awk**, écrire les lignes de commande permettant de :
 - a. Calculer et afficher le montant total des ventes par catégorie de produit et le montant total des ventes pour toutes les catégories.

Correction :

```

awk -F"," '
{
    total_vente = $4 * $5
    ventes[$2] += total_vente
    total_global += total_vente
}
END {
    for (categorie in ventes) {
        print "Catégorie :", categorie, "Montant total :", ventes[categorie]
    }
    print "Montant total des ventes : ", total_global
}' ventesSupermarche.dat

```

- b. Afficher le nombre de transactions réalisées par carte bancaire et le nombre de transactions en espèces.

Correction :

```

awk -F"," '
{
    if ($6 == "carte") carte++
    if ($6 == "espèces") especes++
}
END {
    print "Transactions par carte : ", carte
    print "Transactions en espèces : ", especes
}' ventesSupermarche.dat

```

3. En utilisant **sed**, écrire les lignes de commande permettant de :
 - a. Remplacer le séparateur de colonne par le symbole « | » et convertir toutes les dates au format AAAA-MM-JJ.

Correction :

```
sed -e 's/,//g' -e 's|([0-9]\{2\})\|([0-9]\{2\})\|([0-9]\{4\})\|3-2-1|g' ventesSupermarche.txt
```

- b. Supprimer toutes les lignes vides.

Correction

```
sed '/^$/d' notes.txt
```

Exercice 3 (8 pts)

Nous avons les fichiers suivants qui font partie d'un logiciel de gestion des employés dans une entreprise :

1. `main.c` : Le fichier principal qui utilise les fonctions des modules `employee.c` et `department.c`.
 2. `employee.c` : Implémente les fonctions liées aux employés.
 3. `employee.h` : Déclare les fonctions et structures de données liées aux employés.
 4. `department.c` : Implémente les fonctions liées aux départements.
 5. `department.h` : Déclare les fonctions et structures de données liées aux départements.
 6. `utils.h` : regroupe les fonctions utilitaires (par exemple, des fonctions de validation des données) nécessaires pour tous les fichiers sources.
1. Écrire un Makefile qui permet de générer l'exécutable `gestion` en prenant en compte :
 - La cible pour produire `employee.o` à partir de `employee.c`.
 - La cible pour produire `department.o` à partir de `department.c`.
 - La cible pour produire `main.o` à partir de `main.c`.
 - La cible pour produire `gestion` à partir de `main.o`, `employee.o` et `department.o`.

Correction

```
gestion: main.o employee.o department.o
gcc main.o employee.o department.o -o gestion
```

```
employee.o: employee.c employee.h utils.h
gcc -c employee.c
```

```
department.o: department.c department.h utils.h
gcc -c department.c
```

```
main.o: main.c employee.h department.h utils.h
gcc -c main.c
```

2. Ajouter une cible `clean` pour supprimer les fichiers objets et l'exécutable `gestion`.

Correction :

```
clean:
```

`rm -f *.o gestion`

3. Supposons que `employee.h` est modifié, et que tu lances la commande `make`. Quelles sont les commandes qui seront exécutées ?

Correction

- a. Recompiler `employee.c`: `gcc -c employee.c`
- b. Recompiler `main.c` (car il dépend de `employee.h`) : `gcc -c main.c`
- c. Recréer l'exécutable `gestion` : `gcc main.o -L. -lgestion -o gestion`